

Dijkstra's Algorithm

Frans Ekberg

Spring 2025

Introduction

A graph is a data structure that is comprised of a set of nodes linked to each other using connections. These connections can be strung together to make a paths between different nodes. In this report, all connections are two-way connections. In a previous report the goal was to construct a program that could find the shortest path between two cities in a data set of Swedish train routes. In this report, an new algorithm will be implemented to search for the shortest path between two cities in a data set of European train routes. This new algorithm is called Dijkstra's algorithm and should be able to find the shortest route between two cities in a reasonable amount of time. To test this, multiple benchmarks will be set up.

Benchmark

To set up a benchmark, the **time.h** library is used using the following code:

```
clock_gettime(CLOCK_MONOTONIC, &t_start);  
...  
clock_gettime(CLOCK_MONOTONIC, &t_stop);
```

The code starts a clock, runs the code that need to be evaluated and then stops the clock. The time is then recorded and stored. The benchmarks in this report will take a relatively long time compared to the clocks frequency. This means that one measurement will suffice to get an accurate execution time.

Creating the map

To create a graph or map of the given data, some structures need to be set up. Firstly, the data structures from the previous report have to be included. These are the city struct and the connections struct. Then the data can be read and put into a graph the same way as in the previous report. The data is a CSV-file of direct rail connection between 135 cities in Europe.

Each row in the file is read into a buffer, then the string is divided into it's three parts and the following code is run, just like in the previous report:

```
city *src = lookup(trains, strtok(copy, ","));
city *dst = lookup(trains, strtok(null, ","));
unsigned int dist = atoi(strtok(null, ","));

connect(src, dst, dist);
```

The maps is now constructed and work can begin on implementing the new algorithm.

Preparations for the algorithm

To use Dijkstra's algorithm, some preparations need to be made. First and foremost, a new data structure need to be constructed.

The path structure

The structure can be called a **path**. It resembles the **connection** struct, as it contains a destination city, a previous city (origin city) and a time integer. However, the path data structure will be used very differently. The time value stored is not the time value it takes to travel from the prev city to the destination city. Instead, the time value contains the time it takes to travel from the original origin city to the current destination city.

These paths will be put in an array called **done**. This array will contain all the shortest paths from the original origin city to the city in the destination of each path. This means that if the final destination is found, the shortest path can be traced back to the origin city using the prev city pointers. The shortest path from the origin city to each city visited will be stored in the array at the location of the destination city ID. For example, if we start in *Göteborg* and the algorithm finds the shortest path to *Stockholm*, which has the ID of 13, then that path will be stored at the 13th place of the **done** array.

The priority queue

The shortest paths are stored in the **done** array, but to get the shortest paths, a second data structure is needed. The algorithm needs a priority queue that can hold paths. The implementation chosen for this project was the array heap implementation of a priority queue. The array was reconstructed to be able to hold paths, and the path with the highest priority was the path with the smallest time variable.

When the priority queue had been implemented and worked, the implementation of the algorithm could begin.

Dijkstra's Algorithm

The algorithm implements a kind of breath-first search to find the shortest path. In the previous report, a deep-first search was implemented, this meant that some searches took a lot of time, and some searches were not possible in a reasonable amount of time. The BFS approach of Dijkstra's algorithm should make it much quicker and more capable. The algorithm goes as follows:

- Take the first path of the priority queue
- If the path leads to the final destination, the search is done and this path is the shortest path
- If not, check if this city has been visited before, by looking in the done array
- If it has, this is a longer path to this city, restart the algorithm
- If it hasn't, this is the shortest path to this city, store it in the done array
- Take all the connections from this city and make new paths, then put each of these new paths back into the priority queue
- Continue until the final destination is found

For this algorithm to work, it needs a few things to be true at all times. Firstly, the next shortest path on the whole map needs to come out of the priority queue at all times. This is why the priority queue puts the shortest paths in the front of the queue. Second, the path to a city in the done array needs to be the shortest path from the origin to that city. Otherwise the algorithm cannot be sure that it has found the shortest path to the destination. This is why the cities are searched from the origin and then the shortest route is explored one at a time. This ensures that the shortest path

from the origin to any city is the only ones save in the done array. These two "absolute truths" are crucial for the algorithm to work.

One problem that was encountered that led to loops in the program was that each time a node was dequeued, even if it wasn't the shortest path to any city, its connections were added back into the p-queue. This led to much wasted memory and time. This was fixed before doing the following benchmarks.

Benchmarks and Time complexity

The table below shows some benchmarks from **Table 3** (DFS) in the previous report and compares them to benchmarks of the current algorithm (Dijkstra's) Doing the same benchmarks as in gives the results following results, see *Table 1* below.

From	To	Path	DFS (ms)	Dijkstra's (ms)
Malmö	Göteborg	153	0.18	0.04
Malmö	Helsingborg	48	0.63	0.01
Stockholm	Göteborg	211	0.10	0.05
Göteborg	Stockholm	211	0.12	0.04
Sundsvall	Umeå	190	1590.55	0.01
Umeå	Sundsvall	190	0.00	0.00
Malmö	Kiruna	1162	706.46	0.24
Kiruna	Malmö	1162	0.80	0.13
Stockholm	Hamburg	407	null	0.08
Stockholm	Valencia	1449	null	0.19
Narvik	Valencia	2528	null	0.21
Valencia	Narvik	2528	null	0.22
Narvik	Taranto	2788	null	0.25
Taranto	Narvik	2788	null	0.14
Edinburgh	Taranto	1733	null	0.14
Taranto	Edinburgh	1733	null	0.20
Edinburgh	Narvik	2324	null	0.23
Narvik	Edinburgh	2324	null	0.25

Table 1: Execution time of finding the shortest path between a number of cities using Depth-First Search (DFS) and Dijkstra's algorithm

The performance is greatly improved using Dijkstra's algorithm compared to DFS. Even when the paths are much longer Dijkstra's performance doesn't seem to suffer that much. The order of the connections that caused problems in the DFS is no factor in Dijkstra's, as the all connections end up in the priority queue anyway.

To determine the time complexity of Dijkstra's algorithm, the execution time can be compared to the number of paths in the **done** array. This gives a somewhat accurate measurement of the time complexity. Doing so gives the following results see *Figure 1* below.

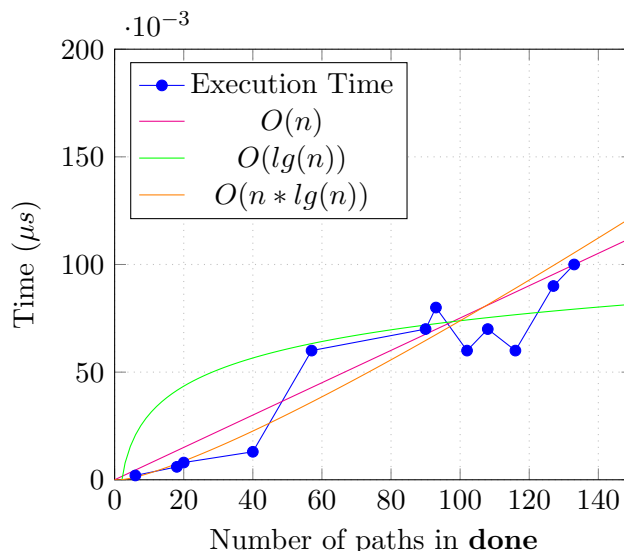


Figure 1: Execution time Dijkstra's Algorithm

The results are a little inconclusive. It is hard to say if the data portrays a linear execution time or a logarithmic. The algorithm could even have an execution time of $O(n * \lg(n))$. By looking at the different operations in the algorithm, we can get more information. The queue functions should have a logarithmic time complexity. The hash lookup has a time complexity of $O(1)$ and the algorithm itself moves through elements pretty fast. My guess is that Dijkstra's Algorithm has a time complexity of $O(n * \lg(n))$, as it has to find all the elements in the done array and take them from the p-queue. This means that it will handle all the elements at least once, and put all the connections of each element into the p-queue at least once.

In conclusion, regardless of the time complexity, the Dijkstra algorithm is shockingly good at finding the shortest distance between two nodes in a graph in a short amount of time.